William Anderson

Senior Software Engineer

Seminole, FL | williamandresondev@outlook.com | +1 (407) 801-1287 |
https://www.linkedin.com/in/william-andreson-39b88b3aa

Professional Summary

Senior Software Engineer with 10+ years of experience building **SaaS**, workflow, and customer-facing web platforms across product, operations, and service-driven industries, specializing in **React 16–19**, **Next.js 9–15**, **Node.js 10–20**, and **NestJS 7–10** to deliver secure, high-performance applications with modern API architecture, scalable frontend systems, and production-ready cloud delivery. Strong background in solving complex bugs, modernizing legacy codebases, improving release reliability, and shipping features that raise usability, stability, and delivery speed across fast-moving engineering teams.

Technical Skills

- **Frontend** — **React 16–19**, **Next.js 9–15**, TypeScript, JavaScript, HTML5, CSS3, Tailwind CSS, Sass/SCSS, Redux, Context API, React Query, responsive design, accessibility, component architecture, SSR, CSR
- **Backend & APIs** — **Node.js 10–20**, **NestJS 7–10**, Express.js, REST APIs, GraphQL, WebSockets, authentication, authorization, JWT, RBAC, background jobs, API integration, file processing
- **Data & Messaging** — PostgreSQL, MySQL, MongoDB, Redis, Elasticsearch, OpenSearch, RabbitMQ, Kafka, AWS SQS/SNS
- **Cloud & DevOps** — AWS, Docker, Kubernetes, Nginx, GitHub Actions, GitLab CI, CI/CD pipelines, Terraform, CloudWatch, Vercel
- **Testing & Quality** — Jest, React Testing Library, Cypress, Playwright, Supertest, integration testing, API contract validation, performance tuning, incident debugging

Professional Experience

Viaro Networks Inc.

Senior Software Engineer | *January 2021 – December 2025*

- Led development of large-scale product workflows using **React 18**, **Next.js 13–14**, **Node.js 18–20**, and **NestJS 9–10**, delivering customer-facing modules and internal tools that improved feature delivery speed by **34%** across multiple business teams.
- Architected modular frontend and backend patterns that standardized data flow, validation, and shared service contracts, which reduced regression-heavy releases by **29%** and made parallel team delivery significantly easier.
- Resolved a recurring production issue where dashboard pages froze under heavy filter changes, then introduced query caching, request deduplication, and render optimization that improved perceived responsiveness by **41%** for high-usage screens.
- Implemented secure role-aware admin capabilities with **JWT** and **RBAC**, aligning frontend visibility with backend authorization rules so that support and operations teams could work faster without creating access-control gaps.
- Migrated legacy server-rendered account workflows into **Next.js** applications with cleaner routing and reusable UI composition, giving the product a more maintainable codebase and reducing page-level duplication across teams.

- Built scalable backend services in **NestJS** for notifications, reporting, and account events, separating controller, service, and data-access responsibilities to improve testability and reduce debugging effort during releases.
- Fixed a high-impact issue in export generation where duplicated background requests produced inconsistent report files, then added idempotent job handling and queue-based processing that improved reliability by **37%**.
- Introduced structured logging and error monitoring across frontend and backend paths, enabling faster root-cause analysis for intermittent API failures and cutting average issue investigation time by **32%**.
- Delivered real-time activity features using event-driven patterns and WebSocket updates, allowing users to see status changes faster while reducing manual refresh behavior across operational screens.
- Optimized PostgreSQL query paths for reporting and search endpoints through indexing, payload trimming, and better pagination, improving backend response times by **35%** on several critical endpoints.
- Supported cloud deployment improvements with **AWS**, **Docker**, and CI/CD automation, helping the team reduce release friction, stabilize promotion flows, and make rollback handling far more predictable.
- Partnered closely with product and design teams to ship complex new features iteratively, balancing urgent business asks with maintainable engineering decisions so the platform could evolve without accumulating fragile shortcuts.

VividCloud

Software Engineer | *January 2016 – December 2020*

- Built modern web applications with **React 16–17**, **Next.js 9–10**, **Node.js 12–14**, and later **NestJS 7–8**, helping the company transition from simpler frontend delivery patterns into more structured full-stack product development.
- Developed reusable UI modules for dashboards, forms, account management, and workflow-heavy administration, which improved consistency across screens and reduced repeated implementation effort by **27%**.
- Contributed to backend API development for customer data, reporting, and workflow processing, introducing cleaner route organization and stronger validation that reduced avoidable support issues during releases.
- Solved a persistent bug where stale frontend state caused users to overwrite newer values after long editing sessions, then added refresh safeguards and state reconciliation logic that improved data accuracy by **22%**.
- Helped modernize earlier Node services into **NestJS**-based modules with clearer dependency boundaries, improving maintainability and giving the team a stronger foundation for feature expansion and test coverage.
- Implemented file upload and processing flows for operational datasets, adding validation, failure messaging, and safer backend handling so teams could recover from malformed inputs without manual engineering intervention.
- Improved application performance by reducing oversized API responses, optimizing client rendering paths, and tightening search queries, which lowered load times by **31%** on several internal reporting views.
- Added authentication and permission-aware route handling across both frontend and backend layers, improving usability for different user groups while reducing accidental access to restricted functions.
- Fixed deployment instability caused by environment drift between staging and production, then standardized environment configuration and pipeline checks so releases became more repeatable and less error-prone.
- Worked with **Redis**, relational databases, and background processing to support asynchronous workflows, helping the platform handle heavier operational tasks without blocking synchronous user requests.

- Supported feature launches from planning through production stabilization, regularly handling post-release issues, edge-case bugs, and API mismatches in ways that improved delivery confidence across the engineering team.

Minimalist Moon
Software Developer | *January 2013 – December 2015*

- Developed business web applications using **React 0.14–15**, **Node.js 0.12–4**, and **Express 4**, contributing to practical customer and internal workflow tools during an early stage of modern JavaScript adoption.
- Built form-driven interfaces and backend endpoints for account actions, approvals, record management, and reporting needs, focusing on predictable business behavior and maintainable delivery within a small engineering team.
- Fixed a recurring issue where duplicate form submissions created inconsistent records, then added request guards and safer backend processing rules that reduced data cleanup effort by **24%** for support staff.
- Implemented reusable frontend components and shared utility patterns that reduced repeated page code and made it easier to expand similar workflows without reintroducing old UI inconsistencies.
- Created REST-style service endpoints and database-backed business logic for status handling, reporting, and operational search, giving the company a stronger separation between interface behavior and server processing.
- Improved query efficiency for list and report pages by restructuring SQL, limiting payload size, and introducing practical indexing, which improved page responsiveness by **28%** on commonly used screens.
- Added validation and clearer user-facing feedback for import and export workflows so operations teams could correct bad files faster and complete processing with fewer engineering escalations.
- Helped stabilize production behavior by tracing hard-to-reproduce errors from logs and user reports, then tightening edge-case handling in both frontend and backend layers to reduce repeat incidents.
- Contributed to release packaging, environment setup, and deployment coordination, improving handoff reliability and making it easier for the team to ship features with fewer last-minute configuration issues.
- Supported gradual adoption of more component-based frontend patterns, which gave the team flexibility to improve maintainability without forcing a risky full rewrite of existing application screens.

Microsoft
Software Developer Intern | *July 2012 – December 2012*

- Supported web application development and internal tooling enhancements using widely adopted engineering practices of the period, contributing frontend and backend changes while learning structured delivery, code review, and quality expectations in a large-scale environment.
- Implemented small but meaningful improvements to UI behavior, form validation, and data handling flows, helping reduce user friction and giving senior engineers cleaner foundations for additional feature work.
- Investigated and fixed defects related to inconsistent field validation and edge-case submission handling, improving reliability for internal users while building practical debugging and testing discipline.
- Assisted with backend service updates, data mapping tasks, and integration support work, gaining hands-on experience with maintainable coding standards, team collaboration, and release-oriented engineering practices.

Education

Florida State University
Bachelor's Degree in Computer Science | 2008 – 2012